DS5003A – Data Engineering

Assignment: NumPy, SciPy, File I/O and Visualization

Duration: 2:00 PM – 4:45 PM
Total Questions: 8

Instructions

- Follow all constraints specified in each question strictly.
- Use of prohibited built-in functions will result in loss of marks.
- Write clean, well-commented Python code.
- Use NumPy, SciPy, File I/O, and Matplotlib as required.
- Please call the TA once you have finished the assignment for marking and the mini-viva.

Question 1: Temperature Analysis Using NumPy

A weather station records the temperature for 10 consecutive days. Write a Python program to analyze the recorded temperatures using NumPy.

Input

```
Temperatures:
[31, 29, 34, 28, 35, 32, 30, 36, 27, 33]
```

Calculate the average temperature, hottest temperature, coldest temperature, and all temperatures above the average.

Expected output

```
Average temperature: 31.5
Hottest temperature: 36
Coldest temperature: 27

Temperatures above average:
34 35 32 36 33
```

Constraints

- Store the temperatures in a NumPy array.
- Use NumPy aggregation functions where appropriate.
- Use Boolean indexing to find temperatures above the average.
- Do not use Python's built-in `max()` or `min()` functions.

Question 2: Branch-wise Sales Analysis

A company records the number of products sold by three branches over four weeks.

Input

```
Sales:
[[120, 135, 128, 142],
 [150, 148, 160, 155],
 [110, 125, 118, 130]]
```

Each row represents a branch and each column represents a week.

Write a Python program to calculate the total sales of each branch and each week. Identify the branch and week with the highest sales.

Expected output

```
Branch-wise total:
Branch 1: 525
Branch 2: 613
Branch 3: 483

Week-wise total:
Week 1: 380
Week 2: 408
Week 3: 406
Week 4: 427

Highest selling branch: Branch 2
Highest selling week: Week 4
```

Constraints

- Represent the data using a two-dimensional NumPy array.
- Use the appropriate `axis` argument for aggregation.
- Do not calculate totals using manually written loops.
- Do not use Python's built-in `sum()` function.

Question 3: Employee Sales File Processing

A text file named `employees.txt` contains employee names and their monthly sales.

Input

```
Alice 45000
Bob 32000
Charlie 58000
David 27000
Eva 51000
```

Write a Python program to read the file and calculate the total and average sales. Then create a new file containing only the employees whose sales are above the average.

Expected output

```
Total sales: 213000
Average sales: 42600

Employees above average:
Alice 45000
Charlie 58000
Eva 51000
```

The output file should contain:

```
Alice 45000
Charlie 58000
Eva 51000
```

Constraints

- Read the input file using file handling operations.
- Process the file line by line.
- Do not hard-code the employees above the average.
- Create a separate output file for the filtered records.

Question 4: Student Result Analysis from CSV

A CSV file named `student_marks.csv` contains the assignment and examination marks of students.

Input

```
Student_ID,Assignment,Exam
S101,78,82
S102,65,71
S103,90,88
S104,72,69
S105,85,94
```

The final score is calculated as:

$$\text{Final Score} = 0.40(\text{Assignment}) + 0.60(\text{Exam})$$

Write a Python program to calculate the final score for every student and display students whose final score is at least 75.

Expected output

```
Student   Final Score
S101      80.4
S103      88.8
S105      90.4

Highest scorer: S105
```

Constraints

- Read the CSV data using NumPy.
- Calculate the final scores using NumPy operations.
- Use Boolean filtering to identify eligible students.
- Do not manually check each student's score.

Question 5: Delivery Time Statistical Analysis

A delivery company records the delivery time, in minutes, for ten orders.

Input

```
Delivery times:
[32, 45, 28, 51, 37, 42, 29, 95, 34, 41]
```

Using `scipy.stats`, calculate the mean, median, mode, and z-score for each delivery time.

Identify the delivery time having the highest z-score.

Expected output

```
Mean: 43.4
Median: 39.0
Mode: 28

Highest z-score value: 95
```

Constraints

- Use `scipy.stats` for statistical calculations.
- Calculate z-scores using an appropriate SciPy function.
- Do not manually implement the z-score formula.
- Identify the unusual value programmatically.

Question 6: Attendance and Marks Correlation

A teacher wants to determine whether student attendance is related to examination marks. Consider the following **Input**

```
Attendance:
[65, 72, 78, 80, 85, 88, 92, 95]

Marks in Physics:
[55, 60, 68, 70, 76, 79, 85, 91]

Marks in English:
[76, 78, 73, 65, 89, 99, 95, 94]
```

Using SciPy, calculate the Pearson correlation coefficient. Determine whether the relationship between attendance and marks is positive or negative and whether it is weak, moderate, or strong. Also plot the corelation using heatmap.

Expected output

```
Pearson correlation coefficient attendance to Physics : approximately 0.99
Pearson correlation coefficient attendance to Physics : approximately 0.78
```

Constraints

- Use `scipy.stats` for correlation calculation.
- Do not manually calculate the correlation coefficient.
- Interpret the obtained correlation value.

Question 7: Solve a Business Equation System

A shop sells two types of products, Product A and Product B. The following equations represent the total number of products sold and the total revenue generated:

$$x + y = 80$$

$$50x + 80y = 5600$$

where x represents the number of Product A units and y represents the number of Product B units.

Write a Python program to determine the values of x and y.

Expected output

```
Product A units: 40
Product B units: 40

Equation 1 verified: True
Equation 2 verified: True
```

Constraints

- Represent the equations in matrix form.
- Use `scipy.linalg.solve()` to solve the equations.
- Do not manually calculate the values of x and y.
- Verify the obtained solution using both equations.

Question 8: Business Revenue and Profit Visualization

A small business records its monthly revenue and expenses for six months in 2026.

Input

```
Months:
Jan Feb Mar Apr May Jun

Revenue:
[52000, 58000, 61000, 68000, 72000, 79000]

Expenses:
[42000, 45000, 47000, 50000, 54000, 57000]
```

Write a Python program to calculate the monthly profit and visualize the business performance.

Expected output

```
Monthly Profit:
Jan: 10000
Feb: 13000
Mar: 14000
Apr: 18000
May: 18000
Jun: 22000
```

The program must produce:

1. A line plot showing monthly revenue and expenses.
2. A bar chart showing monthly profit.
3. A scatter plot showing the relationship between revenue and profit.

Further, consider the monthly profits for six months in 2025 and 2024 given below and compare them in the same plot (use appropriate plot type).

Constraints

- Use NumPy for numerical calculations.

Year	Jan	Feb	Mar	Apr	May	Jun
2025	8000	9000	13000	18000	16000	13000
2024	10000	9000	12000	16000	14000	18000

- Use Matplotlib for visualization.
- Do not manually calculate individual profits.
- Add appropriate titles and axis labels.
- Include a legend where appropriate.
- Save the generated visualization to an image file.